

COBRAS User Guide

Installation, data preparation, simulation and analysis

Liam Dauchat

September 16, 2026

Abstract

COBRAS is a Python framework for radial power-grid market and state analysis. This guide explains how to prepare a case, run the built-in methods, inspect their results, save a time-series study, and add a method. It documents the software's actual input conventions and limitations, including distinctions between physical units, per-unit values and squared quantities. The IJEPES manuscript supplies the mathematical derivations; the SoftwareX manuscript describes the research motivation and software architecture. This guide is the practical companion to those papers.

Contents

1	Choose a workflow	2
2	Installation and first run	2
3	Prepare a case folder	3
4	Use the Python API	6
5	Graphical runner	7
6	Time-series studies and persistence	8
7	Interpret results and compare methods	9
8	Plotting and downstream studies	10
9	Extend COBRAS	12
10	Troubleshooting and reproducibility	13
11	Build and maintain this guide	14

1 Choose a workflow

COBRAS operates on one connected radial network with a designated root bus. The built-in methods are a second-order cone programming (SOCP) relaxation of AC optimal power flow, a merit-order dispatch estimator, backward–forward sweep (BFSA) physical-state estimation, and BFSA-based distribution locational marginal price (DLMP) estimation.

The code is distributed under the BSD 3-Clause License; see `LICENSE` and `COPYRIGHT` in the repository. Record the Git revision alongside this guide when reproducing a study.

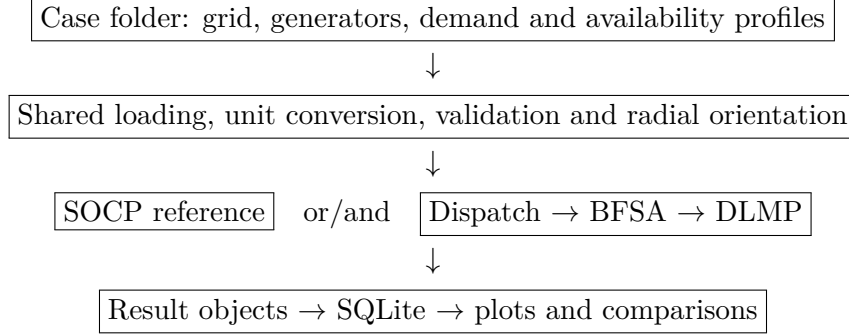


Figure 1: The main COBRAS workflow. A time-series study repeats the method stage for each snapshot. The estimator’s physical state can also seed SOCP.

Task	Starting point
First calculation	Run the estimator CLI in Section 2. No external optimization solver is needed.
Interactive study	Open the solver GUI, select a case folder and choose OPF, estimators or comparison (Section 5).
Repeated snapshots	Use <code>MultiTimestepSolver</code> to write a SQLite database (Section 6).
Explore saved results	Open the plot GUI or call the database plotting functions (Section 8).
Implement a new method	Use the data classes, protocols and registry (Section 9).
Understand the mathematics	See the IJEPES manuscript [1]; see SoftwareX [2] for motivation and design.

The time-series runner solves independent snapshots. It does not introduce intertemporal constraints, storage scheduling, unit commitment or ramp limits. BFSA estimates a state for a dispatch; it does not enforce the network constraints that define the OPF feasible set. A converged BFSA calculation is therefore not a certificate of a feasible or optimal dispatch.

2 Installation and first run

2.1 Python environment

Use Python 3.10 or later. The supplied Conda environment selects Python below 3.13; it is the recommended starting point when reproducing the repository’s research workflows. Clone the repository and create the environment:

```
git clone https://github.com/dauchat-ntnu/COBRAS.git
cd COBRAS
conda env create -f environment.yml
conda activate cobras
```

COBRAS is distributed as a GitHub repository, not as a pip-installable library. Clone the repository first and run the example scripts from its root. On Linux, the graphical interfaces may also require the operating system's Tk package. The dependencies declared in `environment.yml` include NumPy, pandas, NetworkX, PyYAML, tqdm, Pyomo, openpyxl, matplotlib, Plotly and pandapower. Tkinter is used for the GUIs. The scripts import the source files directly from the clone.

2.2 Optimization solver

The estimator pipeline needs no external optimization solver. SOCP runs use Pyomo's solver interface and default to MOSEK. The Conda environment includes the MOSEK interface; configure a valid license separately. Installing the Python interface does not provide a license. An alternative backend must support this Pyomo kernel conic model and the dual values required by your analysis. A solver name accepted by Pyomo is not by itself evidence that it supports this model. Check a small known case before starting a long study. IPOPT is listed in the Conda environment, but the public conic workflow should not be assumed portable to every backend.

2.3 Run the supplied case

```
python examples/run_framework.py --preset estimators --load-index 0
```

The default case is `data/MethodologyPaper/80_bus_base`. The summary should report 80 buses, 79 branches, 2 generators, 32 load records, 80 voltage entries and 80 active-price entries. The CLI defaults to absolute load units (MW/MVAr), selects row zero and prints a summary; it does not create a database. Use an explicit case path when comparing with an archived study:

```
python examples/run_framework.py --preset comparison --case-folder "data/Methodology Paper/80
_bus_base" --load-mode absolute --load-index 0 --opf-solver mosek
python examples/run_framework.py --help
```

Repository data and paper outputs are retained under `data/MethodologyPaper`. The existing databases there are archived results, not the output of the above CLI command. Save new work to a separate output path.

2.4 Check the installation

```
python -m pytest -q -ra
```

Read the skipped-test reasons as well as the pass count. Some tests exercise model construction without a commercial solver; the legacy full-solver tests are skipped. Passing this command alone does not establish that your MOSEK license works, that a GUI has been exercised on your OS, or that every input case is numerically valid. Run the estimator example and a licensed small SOCP case as separate installation checks.

3 Prepare a case folder

3.1 Files and identifiers

The loader accepts MATPOWER-style CSV tables, together with CSV load profiles and an Excel or CSV generator table. It does not expose a direct MATPOWER `.m` file loader. Comma and semicolon separators are detected for the main case tables. Use explicit headers and a dot as the decimal separator.

File	Contents
<code>mpc_base_mva</code>	Positive finite base power in MVA. Plain text containing one number is sufficient. The loader also tries <code>mpc_base_mva.csv</code> ; a named <code>base_mva</code> or <code>BaseMVA</code> column is supported.
<code>mpc_bus.csv</code>	Bus identifiers, types, voltage bounds and optional reference loads.
<code>mpc_branch.csv</code>	Branch endpoints, per-unit resistance/reactance and ratings.
<code>p_load.csv</code> , <code>q_load.csv</code>	Wide load tables. First column contains snapshot labels; remaining columns identify buses.
<code>generators.xlsx</code>	Generator limits, linear price coefficients and optional profile names. A CSV alternative is selected with <code>generators_filename</code> .
<code>profiles.csv</code> (optional)	Generator availability factors; first column contains labels, other columns are profile names.
Bus mapping (optional)	Maps named load columns to numeric bus IDs. Select the file through <code>bus_mapping_filename</code> or the GUI.

Provide the base-power and generator files explicitly in publication studies. For legacy compatibility, a missing base file prints a warning and falls back to 100 MVA; a missing generator file synthesizes a default generator. These fallbacks are conveniences, not substitutes for a complete case description. Branch and generator IDs are assigned from zero-based table row positions; bus IDs come from the bus table. Reordering rows therefore changes branch or generator IDs. All bus, branch and generator IDs must be unique. Branch endpoints and generator/load bus references must exist in the bus table.

3.2 Bus table

Column	Meaning and convention
<code>bus</code> or <code>bus_i</code>	Integer bus ID.
<code>type</code> or <code>bus_type</code>	1 for load, 2 for generator, 3 for root/slack. The first type-3 bus is chosen as root; otherwise the first generator's bus is used. Specify one type-3 bus.
<code>Pd</code> , <code>Qd</code>	MW/MVAr reference maxima used in normalized load mode; default zero. They are not added to the time-series loads.
<code>Vmin</code> , <code>Vmax</code>	Voltage <i>magnitudes</i> in p.u.; defaults 0.9 and 1.1. The loader squares them for the model.
<code>baseKV</code> or <code>base_kV</code>	Base voltage in kV; used in visualization.
<code>Vm</code> , <code>Va</code>	Input voltage magnitude/angle metadata. The built-in SOCP root is fixed to squared voltage 1.0.
Other fields	MATPOWER Area, zone and shunt fields are retained as metadata. Their presence in a table does not imply that each is modeled by the built-in branch-flow formulation.

When constructing `BusData` directly in Python, pass *squared* bounds to `v_min` and `v_max`; for example `v_min=0.9**2`. The loader's `override_voltage_bounds=True` option takes magnitudes in `default_vmin/default_vmax` and squares them consistently.

3.3 Branch table

Required fields are **fbus**, **tbus**, **r** and **x**. Aliases **f_bus**, **t_bus**, **br_r** and **br_x** are supported. Resistance and reactance must already be in p.u. on the case base: the loader does not convert ohms. **status=1** means active; inactive branches do not belong to the oriented tree or SOCP equations. Active parallel branches, cycles and disconnected networks are rejected.

rateA (alias **rate_A**) is read as MVA. The loader sets $\ell_{\max} = (\text{rateA}/S_{\text{base}})^2$. The built-in model enforces this *squared-current* limit. Its separate apparent-power-limit block is disabled; these are not equivalent limits at voltages away from 1 p.u. Supply ratings deliberately. A missing rating uses the legacy value 9999; a zero rating results in a zero current limit, not an unlimited branch. Tap ratio, phase shift and shunt fields are retained but are not implemented as transformer or shunt equations by this SOCP model.

The orientation step mutates a case so each active branch points away from the root. **original_from_bus** and **original_to_bus** retain the input endpoints. Result flow keys follow the oriented direction. A negative flow therefore denotes power flowing toward the root.

3.4 Load profiles and units

```
Date,2,3
t0,2.0,1.0
t1,3.0,1.5
```

The active and reactive files must have the same normalized bus columns and the same ordered snapshot labels. Use finite numeric values; missing or invalid demands must be filled explicitly. Duplicate columns such as 30, 30.1 and 30.2 are summed into bus 30. Avoid dots in named load columns because the suffix is stripped. Multiple loads mapped to one bus are aggregated by the methods. Negative demands represent injections.

Mode	Conversion to solver-facing LoadData
absolute	Entries are MW/MVAr; divide by base_mva . For a 10 MVA base, a 2 MW demand becomes 0.2 p.u.
normalized	Multiply the entry by bus Pd/Qd , then divide by the base power. A factor 0.5 with a 4 MW reference on a 10 MVA base becomes 0.2 p.u.
auto	Legacy inference from nonzero bus Pd/Qd . If normalized mode is not selected, the entries are passed through as p.u., without division by base power.

The Python loader defaults to **auto**, whereas the introductory CLI uses **absolute**. Always state the load mode in a reproducible study. In particular, the legacy auto fallback must not be confused with explicit absolute mode. Auto can infer a different interpretation for different snapshots; prefer an explicit mode for time-series studies.

Use **load_index=0** for the first data row; negative indices are rejected. Use **load_at="t0"** to select a label, which takes precedence if both selectors are supplied. The range runner uses zero-based row indices and includes both endpoints. It does not infer an hourly or subhourly duration from labels.

3.5 Generators and availability

Use lowercase headers **bus**, **pmin**, **pmax**, **qmin**, **qmax**, **status**, **cp**, **cq**, **inflow_profile**. Unlike absolute load entries, generator limits are read directly as p.u.; convert MW/MVAr to p.u. before writing the generator file. Defaults are **pmin=0**, **pmax=1**, **qmin=-1**, **qmax=1**, **status=1**, **cp=1**, **cq=0**. The input loader does not currently read a constant-cost coefficient; **c_0** is not part

of the built-in SOCP objective. Voltage-control fields in a spreadsheet are not generator voltage constraints in the present model.

`inflow_profile` selects a column of `profiles.csv`. A factor scales `p_max_available`, `q_max_available` and `q_min_available`; the static nameplate fields remain available for reporting. Factors must be finite and non-negative. Values above one are allowed for capacity-scaling studies, so the user is responsible for choosing a meaningful profile. The SOCP active minimum remains the static `p_min`; choose compatible minimums when reducing available capacity. Missing profiles fall back to static limits, with a warning for an unknown named column.

Generator profiles must have the same number of rows as the load files. For index selection they are aligned by *row position*, even if their calendar labels differ; some archived cases use this convention. For label selection the requested label must exist in every relevant file. Document any deliberate alignment between different calendars.

4 Use the Python API

4.1 A small example without input files

```
from libs.shared import (BusData, BranchData, GeneratorData,
                        LoadData, PowerFlowCase)
from libs.methods import EstimatorPipelineSolver

case = PowerFlowCase(
    buses=[BusData(1, bus_type=3, v_min=.9**2, v_max=1.1**2),
           BusData(2, v_min=.9**2, v_max=1.1**2)],
    branches=[BranchData(0, 1, 2, r=.01, x=.02, rateA=10)],
    generators=[GeneratorData(0, 1, p_max=2, q_max=2,
                              c_p=20, c_q=2)],
    loads=[LoadData(2, p_d=.2, q_d=.05)],
    root_bus=1, base_mva=10,
)
solver = EstimatorPipelineSolver(
    physical_options={"tol": 1e-6, "max_iterations": 100})
result = solver.solve(case)
assert result.convergence_info["converged"]
print(result.voltages[2] ** .5) # voltage magnitude
print(result.flows[(1, 2)])    # P and Q in p.u.
print(result.duals_p)          # estimated active DLMPs
```

`EstimatorPipelineSolver.solve` returns the legacy `OPFResult` used by persistence and plotting. It runs dispatch, physical and economic stages through a shared execution function and includes all three stages in its wall time. The public `BFSA` physical and `DLMP` classes expose their stages separately. The combined `BFSASolver.solve` remains a compatibility API.

4.2 Load files and run individual methods

```
from pathlib import Path
from libs.shared import load_case_from_folder, orient_radial_network
from libs.methods import build_method

case = orient_radial_network(load_case_from_folder(
    Path("data/Methodology Paper/80_bus_base"),
    load_index=0, load_interpretation_mode="absolute"))
dispatch = build_method("dispatch", "dummy_merit_order").estimate(case)
physical = build_method("physical", "bfsa").estimate(case, dispatch=dispatch)
economic = build_method("economic", "bfsa_dlmp").estimate(
    case, physical, dispatch=dispatch)
```

```
# Requires an installed and licensed solver:
opf = build_method("opf", "socp", {"solver_name": "mosek"})
reference = opf.solve_optimization(case, warm_start=physical)
```

`solve_optimization` returns `OptimizationResult`; `SOCPSolver.solve` returns `OPFResult`. Warm starts supply initial values, not constraints and not a guarantee of reduced solution time. The backend decides whether it can use those values.

4.3 Options and their effects

Option	Meaning
<code>solver_name, solver_options</code>	Pyomo backend and its backend-specific options.
<code>allow_load_shedding</code>	Adds separate active/reactive nodal curtailment variables to SOCP. Defaults to false. Penalties are configurable on <code>SOCPSolver</code> .
<code>tol=0.001</code>	BFSa relative convergence tolerance on successive squared voltages and/or squared currents.
<code>max_iterations=100</code>	BFSa iteration cap. Inspect the returned convergence flag.
<code>convergence_check="both"</code>	May be <code>voltage</code> , <code>current</code> or <code>both</code> .
<code>compute_losses=True</code>	Iterates electrical losses. False selects a single approximate pass.
<code>v_root_sq=1.0</code>	BFSa root squared voltage. Keep it consistent with SOCP when comparing methods.
<code>merit_order_mode="mcp"</code>	Use explicit active/reactive price seeds. Supply both <code>mcp_active_price</code> and <code>mcp_reactive_price</code> ; this changes the price seed, not the merit-order dispatch.
Plot flags	<code>show_net_bus_plot</code> and <code>show_merit_order_plot</code> display BFSa diagnostic figures.

Defaults are maintained in `config/default.yaml`, read once per process. Restart Python after editing that file. Compatibility fields named `loss_feedback_*` do not enable redispach: the current physical estimator reports losses without changing the clearing generator. Do not use them as controls for an iterative market-clearing algorithm.

5 Graphical runner

Start the GUI with `pythonexamples/run_framework.py`. Its data directory defaults to `data/MethodologyPaper`. To use another collection of case folders, set the `COBRAS_DATA_DIR` environment variable before launching:

```
# PowerShell
$env:COBRAS_DATA_DIR = "C:/my-cases"
python examples/run_framework.py
```

The directory should contain one subfolder per network. The input-file selectors let you choose a bus table, branch table, demand files, generator file, base-power file and optional bus mapping. Select the load interpretation explicitly, then choose a snapshot or an inclusive range.

Choose OPF, estimator pipeline or comparison. Configure the backend and shedding options for OPF; configure tolerance, iteration limit and price mode for the estimator. Set the output directory before starting a calculation. Run launches a Python subprocess and records a log

under `log/`; inspect the log for solver progress or errors. The GUI remains responsive, and Kill requests termination of the launched process. Results from an interrupted run may be incomplete; verify the database timestep coverage before plotting. The process launch no longer depends on a PowerShell executable. Native GUI behavior must still be tested on each supported operating system.

The GUI exposes the built-in method combinations. Registry extension alone does not automatically generate every method-specific option control; new methods may need GUI wiring in addition to registration.

6 Time-series studies and persistence

6.1 Run a range

```
from pathlib import Path
from libs.methods import EstimatorPipelineSolver
from libs.shared.multi_timestep_solver import MultiTimestepSolver

runner = MultiTimestepSolver(
    input_folder=Path("data/Methodology Paper/80_bus_base"),
    output_db=Path("results/tutorial/study.db"),
    case_config={"load_interpretation_mode": "absolute"},
    solvers={"bfsa": EstimatorPipelineSolver()},
)
summary = runner.run_range(0, 2, verbose=False)
print(summary["timesteps_completed"]) # 3, endpoints included
print(summary["db_path"])
```

Choose a new output database name for each study. `TimestepDatabase.create_tables` overwrites an existing database and its matching dual workbook. The runner does not resume a previous run.

`case_config` accepts the loader filename keys, for example `generators_filename`, `profiles_filename`, `mpc_base_mva_filename`, `p_load_filename` and `q_load_filename`, as well as load interpretation and voltage overrides. Static tables and time-series frames are cached. Each returned case owns its bus and branch records so changing one snapshot does not change another.

To compare methods, add `"socp": build_method("opf", "socp")` to `solvers`. Use `warm_start_mode="Estimator"` for a same-snapshot estimator seed, or `"ACOPFt-1"` for a previous successful OPF result. `warm_start_solver_name` selects the estimator explicitly when needed. An estimator used both for comparison and a warm start runs only once per snapshot. After a skipped infeasible step, the previous-result seed is the last successful result, which need not be from the immediately preceding row.

With `continue_on_socp_infeasible=True` and shedding disabled, recognized SOCP infeasibility is recorded and processing continues. Other failures stop the run. `timesteps_completed` counts processed row indices, including rows containing a solver failure; it is not a count of feasible OPF results.

6.2 Database contents and units

Table	Purpose
Grid_Buses, Grid_Branches, Grid_Generators	Static network and generator records.
Grid_Metadata	Base power and root bus.

Table	Purpose
Res_Timesteps	Solver, index, scenario label, cost, timing, convergence status and selected run settings.
Res_Buses	Squared voltage, active/reactive demands and active/reactive prices.
Res_Branches	Oriented active/reactive power flows and squared current <code>ell</code> .
Res_Generators	Active/reactive dispatch and available active capacity.
Res_SOCPDuals	Named constraint duals where available.
Res_SolverTimingSummary	Aggregate wall, build, optimization and extraction timings.
Agg_BusMetrics, Agg_Branch Metrics	Range aggregates.

Bus voltage is stored as V^2 , not V ; current is stored as I^2 , not I . Power flows, demands and dispatch remain p.u. Multiply power by `base_mva` for MW/MVAr. A result's `voltages`, `flows`, `generator_output`, `duals_p/duals_q` and `branch_currents` use those same conventions.

`Res_Buses.load_shed` stores active curtailment when supplied by the solver, and `Res_Branches.flow_loss` stores active losses from BFSa diagnostics or resistance times `ell`. Archived databases produced before this review contain zero placeholders in these fields; they must not be used to infer an absence of shedding or losses. Reactive curtailment and SOCP total curtailment remain in the in-memory convergence diagnostics. BFSa line losses are also available in `line_state_bfsa`.

Dual workbooks are exported on closing a write session and large sheets are split at Excel's row limit. CSV export is available through `runner.export_results_to_csv` or `TimestepDatabase.export_to_csv`. The database is the primary artifact; preserve the source input folder, configuration, software revision and solver versions beside it. Not every method option or original timestamp is stored in the schema.

6.3 Read a stored metric

```
from libs.shared.timestep_database import TimestepDatabase
db = TimestepDatabase("results/tutorial/study.db")
rows = db.get_bus_timeseries(2, 0, 2, "bfsa", "voltage")
print([(t, v_sq ** .5) for t, v_sq in rows])
```

Range-result retrieval combines statistics over a horizon; it is not itself a physically solved snapshot. The runner retains result objects in memory as well as writing SQLite. For very long horizons, run manageable batches with distinct output files and account for this memory use.

7 Interpret results and compare methods

7.1 Validity before accuracy

Check convergence and solver termination, finite voltages, expected bus and branch counts, and the input units. Inspect voltage and current constraints and the conic gap

$$v_i \ell_{ij} - p_{ij}^2 - q_{ij}^2.$$

A small non-negative gap supports tightness of the relaxation at that branch; convergence alone does not establish SOCP exactness. The BFSa implementation clips very low squared voltages

internally. Severe undervoltage and divergent cases need independent feasibility checks; do not accept a plotted state solely because it has finite values.

BFSA holds the root voltage fixed and does not redispatch generation to cover network losses. Its reported generator outputs can therefore differ from the root injection implied by the electrical solution. Positive minimum generator outputs and reactive absorption are not enforced by the simple merit order. When supply is insufficient, it includes a synthetic zero-cost load-excess offer; this is not an economically meaningful scarcity-price model.

7.2 Costs, prices and timing

The SOCP model minimizes linear generation cost plus any shedding penalties. Reported cost is the p.u. objective multiplied by base power. If coefficients are in currency/MWh and the snapshot is interpreted as one hour, the value is a one-hour cost; for other durations apply the duration explicitly. The runner sums snapshot values without multiplying by hours.

The dummy dispatch instead reports clearing price times total demand, separately for active and reactive power. This is a settlement-style value, not the SOCP production-cost objective. An automatic difference between these fields is not an optimality gap. Compute a common cost functional on both dispatches when making that comparison.

Missing or rejected SOCP balance duals are represented as NaN rather than zero. Their absence is not evidence of a zero marginal price. The BFSA prices are estimates based on its seed and analytical propagation assumptions. They are not congestion duals of a constrained dispatch optimization.

For runtime comparisons distinguish the external solver’s solve time from total Python wall time. SOCP records build, optimize, extract and total times; the estimator adapter records all three stages. Include warm-up policy, hardware, backend threads, horizon, repetitions and loading costs when reporting speedups. The speed-analysis script excludes file loading from its per-solve wall-time measurement.

7.3 Comparison APIs

`comparison/state_comparison.py` compares `DispatchResult`, `PhysicalStateResult`, `EconomicStateResult` and `OptimizationResult`. `comparison/benchmark.py` supports legacy `OPFResult` pairs, sweeps and publication tables. These two APIs serve different callers; select one deliberately. The framework signed percentage is $100(x_{\text{candidate}} - x_{\text{reference}})/|x_{\text{reference}}|$. Maps are compared on common keys; missing keys are not automatically a failed test. At a zero reference, a nonzero candidate yields an infinite percentage which is omitted by finite-value summary statistics. Read the count together with the error. Legacy benchmark and plot paths have different zero-reference and small-flow handling, so do not assume their aggregates are interchangeable.

8 Plotting and downstream studies

8.1 Database plot GUI

```
python examples/generate_plots.py
```

Choose a database, an output folder, solver names, metric, entity IDs and an inclusive index range. IDs may be comma-separated or ranges, such as `2,5,8-12`. Use the IDs in the saved database. Network and merit plots may need the matching original case folder in addition to the database.

Plot type	Use
Bus, branch	Time series of prices, squared voltages, demands or oriented branch flows.
Generator	Active dispatch in p.u. or MW, with availability context.
Box	Distributions grouped by bus or branch and method.
Duration	Descending sorted values or errors; the horizontal position is rank/exceedance, not chronological time.
Solve time	Solver timing by snapshot, including BFSa iteration counts where available.
Dashboard	Bus and branch metrics on a shared time axis.
Network	Bus colors for price or voltage and branch colors for flow/loading; single, difference or side-by-side views.
Merit	Active or reactive offers and clearing point.
Comparison table	Detailed or aggregate differences and timing as CSV.
LMP statistics	Per-bus distribution summaries as CSV.
P-Q case counts	Classification counts exported to Excel.

Several difference modes use the literal solver names `bfsa` and `socp`; preserve those keys when using the built-in comparison GUI. Percent differences are signed BFSa minus SOCP relative to the absolute SOCP reference. Raw differences retain the metric's units. The option to exclude invalid SOCP snapshots also masks the corresponding BFSa observations; report how many observations remain. An empty plot may reflect filtering, an absent metric, a solver-name mismatch or a range without successful results. HTML plots are interactive; the Plotly toolbar can export a high-resolution PNG. Network plotting uses `pandapower` to build a plotting representation, not to validate COBRAS's numerical solution.

8.2 Plot from Python

```
from pathlib import Path
from libs.visualization import create_bus_timeseries_plot
fig, saved_path = create_bus_timeseries_plot(
    "results/tutorial/study.db", bus_ids=[2, 3],
    solver_names=["bfsa"], metric="lmp_p",
    output_dir=Path("results/tutorial"), file_stem="prices")
print(saved_path)
```

8.3 Other entry points

Script	Role and preparation
<code>examples/analyze_load_profiles.py</code>	Raw load summaries, boxplots and optional time series. Run <code>--help</code> for range/export options. Raw profiles retain their input units or factors.
<code>examples/run_speed_analysis.py</code>	Repeated BFSa/SOCP timing. Use <code>--bfsa-only</code> when no optimization backend is available.
<code>examples/run_sensitivity.py</code>	Generator reactive/active cost-ratio sweep. The reusable <code>run_cq_cp_sweep</code> accepts a network folder, generator ID, ratios, snapshot labels, file configuration and solvers. The script's research-specific main block needs configuration.

Script	Role and preparation
<code>examples/plot_sensitivity_lmp.py</code>	Reads a sensitivity run and produces LMP-error boxplots; supply the matching run directory and labels.
<code>libs/methods/economic/full_KKT</code>	Python API for the explicit radial KKT system; exposes its matrix, labels, solution and price diagnostics.

```
python examples/run_speed_analysis.py --bfsa-only --runs 3 --start-index 0 --end-index 2 --
output-dir results/tutorial/speed
```

The speed script currently calls the loader in legacy `auto` mode. Match that convention before comparing its numerical outputs to a study run in explicit absolute mode. Its timing results alone do not establish that two studies used the same physical demands. The full-KKT implementation is under `libs/methods/economic/full_KKT`. It is a diagnostic workflow, not a registered replacement for BFSA-DLMP. Inspect its rank, solution mode and stationarity residual. A least-squares solution of a rank-deficient system is not proof of unique economic prices. The matrix is dense, so this workflow has different memory and scaling properties from the radial sweep. The scenario-creation script removed from this checkout is not part of the supported workflow described here.

9 Extend COBRAS

9.1 Architecture and contracts

Location	Responsibility
<code>libs/shared/data.py, network.py, io.py</code>	Case records, validation, radial topology and input conversion.
<code>libs/shared/results.py, interfaces.py</code>	Result containers, compatibility adapters and method protocols.
<code>libs/methods/base.py, registry.py, defaults.py</code>	Method metadata, lookup/building and YAML defaults.
<code>libs/methods/execution.py, pipeline.py</code>	Shared dispatch-physical-economic execution and the legacy solver adapter.
<code>libs/methods/opf/socp</code>	Model construction, cached parameter updates, backend solve and dual extraction.
<code>libs/methods/physical/bfsa</code>	Electrical sweeps and shared analytical price-propagation primitives.
<code>libs/methods/dispatch, economic</code>	Dispatch and economic estimators, including the separate KKT diagnostic.
<code>libs/shared/gui_runner.py, gui_method_config.py</code>	Interactive configuration and serialized job execution.
<code>libs/shared/multi_timestep_solver.py, timestep_database.py</code>	Horizon orchestration and persistence.
<code>libs/visualization, libs/analysis, comparison</code>	Plotting, load-profile analysis and comparisons.
<code>examples, tests, config</code>	Entry points, regression tests and method defaults.

Use the family that matches your method: `dispatch`, `physical`, `economic` or `opf`. Implement the corresponding protocol in `libs/shared/interfaces.py` and return the matching

result container. Do not reimplement input parsing, topology orientation, SQLite writing or comparison statistics within a method.

9.2 Register a method

This skeleton shows the registration mechanics; replace the estimator with a scientifically meaningful implementation before using it in a study.

```
from libs.methods.base import MethodSpec
from libs.methods.registry import METHOD_REGISTRY, build_method
from libs.shared import EconomicStateResult, MethodCapabilities

class ConstantPrice:
    name = "constant_price"
    capabilities = MethodCapabilities(
        produces_physical_state=False, produces_economic_state=True,
        produces_dispatch=False, supports_warm_start=False,
        requires_physical_state=True)
    def __init__(self, price=20.0):
        self.price = price
    def estimate(self, case, physical_state, *, dispatch=None,
        previous_economic_state=None):
        return EconomicStateResult(
            lambda_p={b.bus_id: self.price for b in case.buses},
            lambda_q={b.bus_id: 0.0 for b in case.buses},
            method_name=self.name)

METHOD_REGISTRY["economic"]["constant_price"] = MethodSpec(
    family="economic", key="constant_price", label="Constant price",
    builder=ConstantPrice, capabilities=ConstantPrice.capabilities,
    default_options={"price": 20.0})
method = build_method("economic", "constant_price", {"price": 25.0})
```

In-memory registration is local to that Python process. For a persistent extension, place the implementation in a method-family package and import its specification from the registry. Add defaults to the YAML file when appropriate. Explicit build options override the registered defaults. `option_schema` describes controls; it is not a general runtime validator. Validate your method's options yourself.

The built-in BFSa physical estimator expects merit-order metadata `p_clearing/q_clearing` in its `DispatchResult`; a generic dispatch containing only setpoints does not satisfy that additional contract. BFSa-DLMP similarly requires BFSa line-state diagnostics in the physical result. Protocol compatibility alone therefore does not make arbitrary method combinations valid. The range/database stack expects an object with `solve(case)` returning `OPFResult`; adapt a new OPF's framework result with `opf_result_from_optimization` when necessary.

Before registering a production method, test a tiny known case, missing or invalid inputs, nonconvergence, output units and IDs, and a two-snapshot run. Compare against an independent reference where possible. Preserve existing public adapters until their consumers have been migrated.

10 Troubleshooting and reproducibility

Symptom	Checks
Import error	Activate the Conda environment and run from the GitHub clone's root. PyYAML provides the <code>yaml</code> import.

Symptom	Checks
No GUI networks	Check <code>COBRAS_DATA_DIR</code> and its immediate child case folders.
Solver unavailable/license error	Verify the backend installation and license in the same environment as Python. Try the estimator-only workflow first.
Unexpectedly large/small loads	Compare absolute , normalized and legacy auto ; inspect generator limits, base power and one converted snapshot.
Invalid topology	Check active status, missing buses, duplicate/parallel branches, cycles and disconnected islands.
Snapshot selection error	Indices are zero-based, range ends are inclusive, labels must be unique for label selection.
Missing prices or empty plots	Inspect solver status, missing duals, solver names, selected metrics, index range and masking.
Different numerical results	Match input files, base power, mode, snapshot, tolerance, solver version, thread count, price seeds and software revision.
Unexpected cost gap	BFSA settlement and SOCP production-cost objectives are different quantities.
Slow long run	Check repeated case parsing in custom scripts, dense KKT work, plot/export time and retained result memory. Reuse the cached range loader and solver instance.

For a published study retain the Git revision, environment/package versions, solver version and settings, input files or checksums, selected rows and their duration, load mode, generator-profile alignment, method settings, convergence criteria and output database. Separate numerical accuracy from feasibility, and report excluded observations together with summary errors.

11 Build and maintain this guide

This file is self-contained: no untracked figure directory or external bibliography is needed. With a LaTeX installation, from `docs` run:

```
pdflatex -interaction=nonstopmode -halt-on-error documentation.tex
pdflatex -interaction=nonstopmode -halt-on-error documentation.tex
```

The second pass resolves the table of contents and cross-references. Check the log for missing references and overfull boxes, then inspect the PDF. The finished IJEPES and SoftwareX sources are separate papers; update this guide when APIs or input conventions change. The organization follows the supplied PowerGAMA and JADE examples: setup, model context, explicit data specifications, working examples, outputs and extension instructions.

References

- [1] L. Dauchat, S. F. Myhre and M. Korpås. *Price-Preserving Aggregation Framework and Methods for Radial Distribution Grids*. IJEPES manuscript supplied with COBRAS, `docs/overleaf/IJEPES.tex`.
- [2] L. Dauchat, S. F. Myhre and M. Korpås. *COBRAS: A python toolkit for Convex Optimal-power-flow Based Radial-grid Analysis, Approximations, and Simulation*. SoftwareX manuscript supplied with COBRAS, `docs/overleaf/SoftwareX.tex`. Publication identifiers are intentionally not inferred here.